



Universidad Nacional Autónoma de México
Facultad de Ingeniería



Estructura y Programación de Computadoras (1503)

Profesora: Ariel Adara Mercado Martínez
Semestre 2026-2

Reporte Técnico

Grupo: 05

Alumnos:

Arcos Garduño Itzcoatl Ajax
García Aguilar José Iván
Martínez Chavez Alexis
Martínez Méndez Cedrik Alexis
Motta Reyes Emmanuel Alberto

Índice

1. Introducción	2
2. Arquitectura del Sistema	2
3. Módulos Implementados	2
3.1 Lexer (src/lexer.c)	2
3.2 Parser (src/parser.c)	4
3.3 Tabla de Símbolos (src/symbols.c)	8
3.4 Ensamblador (src/assembler.c)	11
3.5 Encoder IA-32 (src/encoder.c)	14
3.6 Formato Objeto (src/object.c)	15
3.7 Mini Linker (src/linker.c)	16
4. Instrucciones de Compilación	19
5. Instrucciones de Uso	19
6. Casos de Prueba	19
7. Organización del Equipo	20
8. Dificultades Encontradas	20
Parsing de directivas con identificadores	21
Direccionamiento simbólico en memoria	21
Validación contra NASM	21
Conflictos de Git en trabajo en equipo	21
Separación de headers al integrar registros de 8 bits	21
9. Conclusiones	21

1. Introducción

Este proyecto implementa un sistema completo de traducción y construcción de programas para un subconjunto de la arquitectura IA-32. El sistema incluye un ensamblador de una pasada, un ensamblador de dos pasadas, un generador de archivos objeto y un mini linker, todos implementados manualmente en C sin el uso de herramientas automáticas de parsing como Flex o Bison.

El objetivo principal fue comprender internamente como funcionan los ensambladores, los archivos objeto, los enlazadores y la generación de código máquina, implementando cada componente desde cero.

2. Arquitectura del Sistema

El sistema está compuesto por los siguientes módulos que forman un pipeline de traducción, donde la salida de un módulo constituye la entrada del siguiente:

```
ASM
|
Lexer      - tokenización del código fuente
|
Parser     - construcción del AST
|
Assembler  - ensamblado de una o dos pasadas
|
Encoder    - generación de código máquina IA-32
|
Object     - escritura del archivo objeto
|
Linker     - enlazado y generación del binario final
```

3. Módulos Implementados

3.1 Lexer (src/lexer.c)

El lexer lee el archivo fuente carácter por carácter y produce una lista de tokens. Reconoce los siguientes tipos:

- Registros de 32 bits: EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP
- Registros de 8 bits: AL, AH, BL, BH, CL, CH, DL, DH

- Instrucciones: MOV, ADD, SUB, JMP, CALL, RET, MOVZX, IMUL, IDIV, etc.
- Directivas: SECTION, GLOBAL, EXTERN, DB, DW, DD, RESB, RESW, RESD, ORG, EQU
- Números decimales y hexadecimales (prefijo 0x)
- Identificadores y etiquetas
- Operadores: +, -, *, coma, [,], :
- Comentarios: ignorados desde ; hasta fin de línea

Algoritmo de Tokenización (Pseudocódigo)

ESTRUCTURA Token

tipo: INSTRUCCION | REGISTRO | NUMERO | IDENTIFICADOR |
DIRECTIVA | COMA | CORCHETE_ABR | CORCHETE_CER |
OPERADOR | FIN_LINEA | FIN_ARCHIVO

valor: string

línea: entero

FUNCIÓN tokenizar(archivo)

tokens = lista vacía

MIENTRAS no fin de archivo

carácter = leer_carácter() // Incrementa internamente posición_actual

SI carácter == ';' Entonces

MIENTRAS carácter_actual != '\n' y no fin_de_archivo

avanzar_carácter()

FIN_MIENTRAS

CONTINUAR

FIN_SI

SI carácter es espacio o tabulador Entonces

CONTINUAR

FIN_SI

SI carácter == '\n' Entonces

agregar Token(FIN_LINEA, "\n", línea_actual)

línea_actual += 1

CONTINUAR

FIN_SI

SI caracter es letra o '_' o '.' Entonces

 palabra = leer_palabra_acumulada() // Convierte a MAYÚSCULAS automáticamente

 SI palabra en tabla_registros Entonces agregar Token(REGISTRO, palabra)

 SI NO SI palabra en tabla_instrucciones Entonces agregar Token(INSTRUCCION, palabra)

 SI NO SI palabra en tabla_directivas Entonces agregar Token(DIRECTIVA, palabra)

 SI NO Entonces agregar Token(IDENTIFICADOR, palabra)

 CONTINUAR

FIN_SI

SI caracter es dígito Entonces

 numero = leer_numero_acumulada() // Soporta prefijos '0x' hexadecimales

 agregar Token(NUMERO, numero)

 CONTINUAR

FIN_SI

// Operadores y Delimitadores Directos

SI caracter == ',' Entonces agregar Token(COMA, ",")

SI caracter == ':' Entonces agregar Token(COLON, ":")

SI caracter == '[' Entonces agregar Token(CORCHETE_ABR, "[")

SI caracter == ']' Entonces agregar Token(CORCHETE_CER, "]")

SI caracter == '+' o caracter == '-' o caracter == '*' Entonces

 agregar Token(OPERADOR, caracter)

 FIN_SI

FIN_MIENTRAS

 agregar Token(FIN_ARCHIVO, "EOF")

 RETORNAR tokens

FIN_FUNCIÓN

3.2 Parser (src/parser.c)

El parser toma los tokens del lexer y construye una lista de nodos AST. Cada nodo representa una instrucción, directiva o etiqueta con sus operandos.

Modos de direccionamiento soportados:

- Inmediato: MOV EAX, 10
- Registro a registro: MOV EAX, EBX
- Memoria directa: MOV EAX, [1000]
- Base + desplazamiento: MOV EAX, [EBP+4]
- Base + índice: MOV EAX, [EBX+ECX]
- Base + índice escalado: MOV EAX, [EBX+ECX*4]
- Base + índice escalado + desplazamiento: MOV EAX, [EBX+ECX*4+8]

Durante las pruebas se detectaron y corrigieron dos bugs:

- Loop infinito al parsear identificadores como .TEXT en directivas SECTION
- Fallo al parsear referencias simbólicas dentro de corchetes como [resultado_suma]

Algoritmo de Parsing de Instrucciones (Pseudocódigo)

ESTRUCTURA Operando

tipo: INMEDIATO | REGISTRO | MEMORIA

registro_base: string

registro_indice: string

escala: entero

desplazamiento: entero

valor_inmediato: entero

ESTRUCTURA Instruccion

etiqueta: string

mnemonico: string

operandos: lista de Operando

numero_linea: entero

FUNCIÓN parsear(tokens)

instrucciones = lista vacía

MIENTRAS token_actual.tipo != FIN_ARCHIVO

```

        SI token_actual.tipo == FIN_LINEA Entonces
            avanzar_token()
            CONTINUAR
FIN_SI
instr = parsear_linea()
SI instr no es vacía Entonces
    agregar instr a instrucciones
FIN_SI
FIN_MIENTRAS
RETORNAR instrucciones
FIN_FUNCIÓN

FUNCIÓN parsear_linea()
    instr = nueva Instruccion
    instr.numero_linea = token_actual.linea

    // Validar si la línea comienza con una Etiqueta mediante Lookahead
    SI token_actual.tipo == IDENTIFICADOR Y ondear_siguiente_token().tipo
    == COLON Entonces
        instr.etiqueta = token_actual.valor
        avanzar_token() // Consume el IDENTIFICADOR
        avanzar_token() // Consume el COLON ':'
    FIN_SI

    SI token_actual.tipo == DIRECTIVA Entonces
        RETORNAR parsear_directiva()
    FIN_SI

    SI token_actual.tipo == INSTRUCCION Entonces
        instr.mnemonico = token_actual.valor
        avanzar_token()

    SI token_actual.tipo != FIN_LINEA Y token_actual.tipo != FIN_ARCHIVO
    Entonces
        instr.operandos.agregar(parsear_operando())

```

```

    MIENTRAS token_actual.tipo == COMA
        avanzar_token() // Consume la coma
        instr.operandos.agregar(parsear_operando())
    FIN_MIENTRAS
FIN_SI

    RETORNAR instr
FIN_FUNCIÓN

FUNCIÓN parsear_operando()
    op = nuevo Operando

    SI token_actual.tipo == REGISTRO Entonces
        op.tipo = REGISTRO
        op.registro_base = token_actual.valor
        avanzar_token()
    SI NO SI token_actual.tipo == NUMERO Entonces
        op.tipo = INMEDIATO
        op.valor_inmediato = convertir_a_entero(token_actual.valor)
        avanzar_token()
    SI NO SI token_actual.tipo == CORCHETE_ABR Expansion Entonces
        op.tipo = MEMORIA
        avanzar_token() // Consume '['
        parsear_direccionamiento(op)
        demandar_token(CORCHETE_CER) // Consume o lanza error si
no es ']'
    FIN_SI

    RETORNAR op
FIN_FUNCIÓN

FUNCIÓN parsear_direccionamiento(op)
    SI token_actual.tipo == NUMERO Entonces
        op.desplazamiento = convertir_a_entero(token_actual.valor)
        avanzar_token()

```


RETORNAR
FIN_SI

SI token_actual.tipo == REGISTRO Entonces
 op.registro_base = token_actual.valor
 avanzar_token()

SI token_actual.tipo == OPERADOR Y token_actual.valor == "+" Entonces
 avanzar_token()
 SI token_actual.tipo == REGISTRO Entonces
 op.registro_indice = token_actual.valor
 avanzar_token()

SI token_actual.tipo == OPERADOR Y token_actual.valor == "*" Entonces
 avanzar_token()
 op.escala = convertir_a_entero(token_actual.valor)
 avanzar_token()

FIN_SI

SI token_actual.tipo == OPERADOR Y token_actual.valor == "+" Entonces
 avanzar_token()
 op.desplazamiento = convertir_a_entero(token_actual.valor)
 avanzar_token()

FIN_SI

SI NO SI token_actual.tipo == NUMERO Entonces
 op.desplazamiento = convertir_a_entero(token_actual.valor)
 avanzar_token()

 FIN_SI

 FIN_SI

FIN_SI

FIN_FUNCIÓN

3.3 Tabla de Símbolos (src/symbols.c)

Diccionario que asocia nombres con direcciones. Soporta:

- Símbolos locales, globales y externos

- Detección de redefiniciones
- Símbolos pendientes de definición
- Crecimiento dinámico del arreglo interno

Algoritmo de Gestión de la Tabla de Símbolos (Pseudocódigo)

ESTRUCTURA Simbolo

```

nombre:      string
seccion:     string
offset:      entero
es_global:   booleano
es_externo:  booleano
definido:    booleano

```

ESTRUCTURA TablaSimbolos

```

simbolos: mapa<string, Simbolo>

```

FUNCIÓN inicializar_tabla_simbolos()

```

tabla = nueva TablaSimbolos
tabla.simbolos = mapa_vacio()
RETORNAR tabla

```

FIN_FUNCIÓN

FUNCIÓN agregar_simbolo(tabla, nombre, seccion, offset, es_global)

```

simbolo_existente = buscar_simbolo(tabla, nombre)

```

SI simbolo_existente no es NULO Entonces

```

// Error semántico si el símbolo ya se había definido localmente

```

SI simbolo_existente.definido == Verdadero Entonces

```

ERROR "Error Semántico: Símbolo redefinido en la línea: " + nombre

```

SI NO

```

// El símbolo fue registrado por un fixup o directiva previa; se actualiza

```

```

simbolo_existente.seccion = seccion

```

```

simbolo_existente.offset = offset

```

```

simbolo_existente.definido = Verdadero

```

```

simbolo_existente.es_global = es_global

```

```

FIN_SI
SI NO
    // Registro de una nueva entrada en la tabla
    nuevo_simbolo = nuevo Simbolo
    nuevo_simbolo.nombre = nombre
    nuevo_simbolo.seccion = seccion
    nuevo_simbolo.offset = offset
    nuevo_simbolo.es_global = es_global
    nuevo_simbolo.es_externo = Falso
    nuevo_simbolo.definido = Verdadero

    tabla.simbolos[nombre] = nuevo_simbolo
FIN_SI
FIN_FUNCIÓN

```

```

FUNCIÓN buscar_simbolo(tabla, nombre)
    SI nombre existe en tabla.simbolos Entonces
        RETORNAR tabla.simbolos[nombre]
    FIN_SI
    RETORNAR NULO
FIN_FUNCIÓN

```

```

FUNCIÓN marcar_global(tabla, nombre)
    simbolo = buscar_simbolo(tabla, nombre)
    SI simbolo no es NULO Entonces
        simbolo.es_global = Verdadero
    SI NO
        // Si la directiva GLOBAL aparece antes de la etiqueta física
        nuevo_simbolo = nuevo Simbolo
        nuevo_simbolo.nombre = nombre
        nuevo_simbolo.es_global = Verdadero
        nuevo_simbolo.es_externo = Falso
        nuevo_simbolo.definido = Falso
        tabla.simbolos[nombre] = nuevo_simbolo
    FIN_SI
FIN_FUNCIÓN

```

```

FUNCIÓN marcar_externo(tabla, nombre)
    simbolo = buscar_simbolo(tabla, nombre)
    SI simbolo no es NULO Y simbolo.definido == Verdadero Entonces
        ERROR "Error Semántico: El símbolo local " + nombre + " no
puede ser EXTERN"
    FIN_SI

nuevo_simbolo = nuevo Simbolo
nuevo_simbolo.nombre = nombre
nuevo_simbolo.seccion = "UNDEF"
nuevo_simbolo.offset = 0
nuevo_simbolo.es_global = Falso
nuevo_simbolo.es_externo = Verdadero
nuevo_simbolo.definido = Falso
tabla.simbolos[nombre] = nuevo_simbolo
FIN_FUNCIÓN

```

3.4 Ensamblador (src/assembler.c)

Implementa dos estrategias de ensamblado:

Una pasada: lee el archivo una sola vez generando código parcial. Las referencias adelantadas se resuelven mediante fixups al final de la pasada. Cuando se encuentra una referencia a un símbolo no definido, se emite un placeholder de 4 bytes y se registra un fixup que indica donde parchear cuando el símbolo sea resuelto.

Dos pasadas: primera pasada construye la tabla de símbolos completa calculando todos los offsets, segunda pasada genera el código definitivo sin necesidad de fixups ya que todos los símbolos están disponibles desde el inicio.

Algoritmo de Ensamblador de Una y Dos Pasadas (Pseudocódigo)

```

ESTRUCTURA Simbolo
    nombre:      string
    seccion:     string
    offset:      entero
    es_global:   booleano

```

es_externo: booleano
definido: booleano

ESTRUCTURA Fixup

offset_en_codigo: entero
nombre_simbolo: string
tipo: RELATIVO | ABSOLUTO
seccion: string

FUNCIÓN ensamblar_una_pasada(instrucciones)

codigo = buffer vacío
tabla_simbolos = nueva TablaSimbolos
fixups = lista vacía
offset_actual = 0
seccion_actual = ".text"

PARA CADA instruccion en instrucciones

SI instruccion.etiqueta no es vacía Entonces

agregar_simbolo(tabla_simbolos, instruccion.etiqueta,
seccion_actual, offset_actual, Falso)

FIN_SI

SI instruccion.mnemonico == DIRECTIVA Entonces

procesar_directiva(instruccion, &seccion_actual, &offset_actual)

CONTINUAR

FIN_SI

bytes = codificar_instruccion(instruccion, tabla_simbolos)

SI instruccion_hace_referencia_a_simbolo_indefinido Entonces

fixup = nuevo Fixup

fixup.offset_en_codigo = offset_actual +
calcular_offset_operando(instruccion)

fixup.nombre_simbolo =
obtener_nombre_simbolo_referenciado(instruccion)

```

    fixup.tipo = determinar_tipo_reloc(instruccion) // RELATIVO para
saltos, ABSOLUTO para variables
    agregar_fixup_a_fixups
    emitir_bytes_con_placeholder(codigo, bytes, 0x00000000)
SI NO
    emitir_bytes_reales(codigo, bytes)
FIN_SI

offset_actual += tamaño(bytes)
FIN_PARA

// Resolución tardía de Fixups (Backpatching)
PARA CADA fixup en fixups
    simbolo = buscar_simbolo(tabla_simbolos, fixup.nombre_simbolo)
    SI simbolo == NULL Entonces
        ERROR "Error Semántico: Símbolo no definido: " +
fixup.nombre_simbolo
    FIN_SI

SI fixup.tipo == RELATIVO Entonces
    valor = simbolo.offset - (fixup.offset_en_codigo + 4) // Distancia relativa
EIP
SI NO
    valor = simbolo.offset
FIN_SI

inyectar_parche_en_buffer(codigo, fixup.offset_en_codigo, valor)
FIN_PARA

RETORNAR codigo
FIN_FUNCIÓN

FUNCIÓN ensamblar_dos_pasadas(instrucciones)
// --- PRIMERA PASADA: Construcción de Tabla de Símbolos Completa ---

```

```
tabla_simbolos = nueva TablaSimbolos
offset_actual = 0
seccion_actual = ".text"
```

PARA CADA instruccion en instrucciones

```
    SI instruccion.etiqueta no es vacía Entonces
        agregar_simbolo(tabla_simbolos, instruccion.etiqueta,
seccion_actual, offset_actual, Falso)
    FIN_SI

    SI instruccion.mnemonico == "SECTION" Entonces seccion_actual =
instruccion.operandos[0].valor

    SI instruccion.mnemonico == "GLOBAL" Entonces
marcar_global(tabla_simbolos, instruccion.operandos[0].valor)

    SI instruccion.mnemonico == "EXTERN" Entonces
marcar_externo(tabla_simbolos, instruccion.operandos[0].valor)
```

```
    offset_actual += calcular_tamaño_instruccion(instruccion)
FIN_PARA
```

// --- SEGUNDA PASADA: Generación de Código Máquina Plano y
Relocaciones ---

```
codigo = buffer vacío
relocaciones = lista vacía
offset_actual = 0
```

PARA CADA instruccion en instrucciones

```
    bytes = codificar_instruccion(instruccion, tabla_simbolos)

    SI referencia_a_simbolo_externo(instruccion) Entonces
        relocacion = nueva Relocacion
        relocacion.offset = offset_actual +
calcular_offset_operando(instruccion)
        relocacion.simbolo =
obtener_nombre_simbolo_referenciado(instruccion)
```

```
        relocacion.tipo = (es_instruccion_salto(instruccion)) ?  
R_386_PC32 : R_386_32
```

```
        agregar_relocacion_a_relocaciones
```

```
FIN_SI
```

```
    agregar_bytes_al_fichero(codigo, bytes)
```

```
    offset_actual += tamaño(bytes)
```

```
FIN_PARA
```

```
RETORNAR (codigo, tabla_simbolos, relocaciones)
```

```
FIN_FUNCIÓN
```

3.5 Encoder IA-32 (src/encoder.c)

Genera los bytes de código máquina para cada instrucción. Implementa:

- Construcción del byte ModRM: [mod(2)][reg(3)][rm(3)]
- Construcción del byte SIB: [escala(2)][indice(3)][base(3)]
- Escalas soportadas: 1, 2, 4, 8
- Codificación de desplazamientos de 8 y 32 bits
- Codificación de inmediatos de 32 bits en little endian
- Detección automática de cuando se necesita byte SIB
- Soporte completo para registros de 8 bits: AL, AH, BL, BH, CL, CH, DL, DH
- Instrucción MOVZX: extiende registro de 8 bits a 32 bits sin signo (opcode 0F B6)
- MOV reg8, imm8 con opcode correcto 0xB0+r

La correctitud del encoder fue validada comparando la salida byte a byte contra NASM para instrucciones simples, obteniendo resultados idénticos.

3.6 Formato Objeto (src/object.c)

Formato objeto propio identificado con la firma OBJ. Estructura:

```
[ Encabezado ] - firma, version, conteos y offsets  
[ Secciones ] - tabla de .text, .data, .bss  
[ Símbolos ] - tabla de símbolos con flags global/extern  
[ Relocaciones ] - tabla de parches pendientes (R32 y PC32)
```



```
[ Bytes .text ] - codigo maquina generado  
[ Bytes .data ] - datos inicializados
```

El encabezado actúa como índice permitiendo acceso directo a cualquier sección sin lectura secuencial.

Algoritmo de Archivos Objeto(Pseudocódigo)

ESTRUCTURA Encabezado

magic: 4 bytes (e.g., 'D', 'S', 'O', 'B')
version: 1 byte
num_secciones: entero
num_simbolos: entero
num_relocaciones: entero
offset_secciones: entero
offset_simbolos: entero
offset_relocaciones: entero

ESTRUCTURA EntradaSeccion

nombre: 8 bytes fijos
offset: entero
tamaño: entero
tipo: TEXT | DATA | BSS

ESTRUCTURA EntradaSimbolo

nombre: 32 bytes fijos
valor: entero
seccion: entero
es_global: booleano
es_externo: booleano

ESTRUCTURA EntradaRelocacion

offset: entero
indice_simbolo: entero
tipo: R_386_32 | R_386_PC32

FUNCIÓN escribir_objeto(nombre_archivo, codigo, tabla_simbolos, relocaciones)

```

Fichero = abrir_archivo_escritura_binaria(nombre_archivo)
escribir_estructura(Fichero, Encabezado)
escribir_estructura(Fichero, tabla_secciones)
escribir_bloque_bytes(Fichero, codigo.text)
escribir_bloque_bytes(Fichero, codigo.data)
escribir_estructura(Fichero, tabla_simbolos)
escribir_estructura(Fichero, relocaciones)
cerrar_archivo(Fichero)
FIN_FUNCIÓN

```

3.7 Mini Linker (src/linker.c)

Proceso de enlazado en 5 pasos:

1. Carga — lee todos los archivos objeto del disco
2. Fusión — concatena secciones .text, .data y .bss de todos los objetos
3. Tabla global — construye diccionario de símbolos globales con offsets finales
4. Verificación — confirma que todos los símbolos externos están resueltos
5. Relocaciones — aplica los parches R32 y PC32 en el binario fusionado

Algoritmo de Enlazador(Pseudocódigo)

FUNCIÓN linkear(lista_archivos_objeto, salida_binaria)

// 1. Carga e Infraestructura de Mapeo

PARA CADA archivo en lista_archivos_objeto

objeto = leer_y_validar_objeto(archivo)

FIN_PARA

// 2. Fusión Dinámica de Secciones Binarias

seccion_text_global = buffer vacío

seccion_data_global = buffer vacío

tamaño_bss_global = 0

PARA CADA objeto en objetos_cargados

```

offsets_base[objeto].text = tamaño_actual(seccion_text_global)
offsets_base[objeto].data = tamaño_actual(seccion_data_global)

concatenar_buffers(seccion_text_global, objeto.text_bytes)
concatenar_buffers(seccion_data_global, objeto.data_bytes)
tamaño_bss_global += objeto.bss_size
FIN_PARA

// 3. Resolución de la Tabla Global de Símbolos
tabla_global = nuevo MapaSimbolos
PARA CADA objeto en objetos_cargados
    PARA CADA simbolo en objeto.tabla_simbolos
        SI simbolo.es_global == Verdadero y simbolo.es_externo == Falso
        Entonces
            // Ajuste de re-direccionamiento absoluto relativo al nuevo mapa
            coalescente
            offset_final = offsets_base[objeto] + simbolo.valor
            agregar_a_tabla_global(tabla_global, simbolo.nombre, offset_final)
        FIN_SI
    FIN_PARA
FIN_PARA

// 4. Verificación de Cierre y Enlaces Satisfechos
PARA CADA objeto en objetos_cargados
    PARA CADA simbolo en objeto.tabla_simbolos
        SI simbolo.es_externo == Verdadero Entonces
            SI buscar_en_mapa(tabla_global, simbolo.nombre) == NULL
            Entonces
                ERROR "Linker Error: Símbolo no resuelto enlazando
                artefactos: " + simbolo.nombre
            FIN_SI
        FIN_SI
    FIN_PARA
FIN_PARA

```

```

// 5. Aplicación Cohesiva de Relocaciones (Parcheo de Direcciones
Reales)

PARA CADA objeto en objetos_cargados PARA CADA reloc en
objeto.tabla_relocaciones
    simbolo_resuelto = obtener_desde_mapa(tabla_global,
reloc.nombre_simbolo)
    offset_en_binario = offsets_base[objeto] + reloc.offset

    SI reloc.tipo == R_386_32 Entonces
        valor_parche = simbolo_resuelto.offset_final
    SI NO SI reloc.tipo == R_386_PC32 Entonces
        // Distancia EIP corregida según el nuevo desplazamiento global
unificado
        valor_parche = simbolo_resuelto.offset_final - (offset_en_binario +
4)
    FIN_SI

    escribir_palabra_32bits(seccion_text_global, offset_en_binario,
valor_parche)
    FIN_PARA
FIN_PARA

// 6. Generación del Archivo de Memoria Plana Consolidado
BinarioFinal = abrir_archivo_escritura(salida_binaria)
escribir_bloque_bytes(BinarioFinal, seccion_text_global)
escribir_bloque_bytes(BinarioFinal, seccion_data_global)
escribir_bloque_ceros(BinarioFinal, tamaño_bss_global)
cerrar_archivo(BinarioFinal)

RETORNAR BinarioFinal
FIN_FUNCIÓN

```

4. Instrucciones de Compilación

```

make clean
make

```

5. Instrucciones de Uso

```
# Ensamblar (genera archivo objeto)
./assembler -a examples/test_mov.asm -o test.obj
```

```
# Ensamblar con una pasada
./assembler -l -a examples/test_mov.asm -o test.obj
```

```
# Ensamblar y linkear en un paso
./assembler -al examples/test_mov.asm -o test.bin
```

```
# Generar binario y archivo hexadecimal legible
./assembler -al examples/test_mov.asm -o test.bin --hex salida.hex
```

```
# Modo verbose (muestra tokens, AST y código generado)
./assembler -v -al examples/test_mov.asm -o test.bin
```

Hexadecimal de test_mov.asm esperado

ID	DIR
main	1000

INSTRUCCIÓN	TAMAÑO
MOV EAX, 1	5 bytes
MOV EBX, 2	5 bytes
MOV ECX, 3	5 bytes
MOV EDX, 4	5 bytes
MOV EAX, EBX	2 bytes
MOV ECX, EDX	2 bytes
MOV EAX, [1000]	6 bytes
MOV EAX, [EBP+4]	3 bytes
MOV EBX, [EBP+8]	3 bytes
MOV EAX, [EBX+ECX]	3 bytes

MOV EAX, [EBX+ECX*4]	3 bytes
MOV EAX, [EBX+ECX*4+8]	4 bytes
RET	1 byte

DIR	CÓDIGO
1000	B8 01 00 00 00
1005	BB 02 00 00 00
100A	B9 03 00 00 00
100F	BA 04 00 00 00
1014	89 D8
1016	89 D1
1018	8B 05 E8 03 00 00
101E	8B 45 04
1021	8B 5D 08
1024	8B 04 0B
1027	8B 04 8B
102A	8B 44 8B 08
102E	C3

6. Casos de Prueba

Archivo	Descripcion	Resultado
test_mov.asm	Todos los modos de direccionamiento MOV	PASS
test_jumps.asm	Saltos y referencias adelantadas	PASS
test_call.asm	CALL, RET y manejo de pila	PASS

test_extern.asm	Simbolos externos y relocalaciones	PASS
test_sib.asm	Byte SIB con todas las escalas	PASS
test_multimodule_a.asm	Modulo A: define y exporta funciones	PASS
test_multimodule_b.asm	Modulo B: usa funciones externas	PASS
test_reg8.asm	Registros de 8 bits y MOVZX	PASS
Comparacion con NASM	Validacion byte a byte	Identicos
Compilacion limpia	Sin warnings con -Wall -Wextra	0 warnings
test_error.asm	Instrucción no soportada, registro inválido, etiqueta no definida	12 errores detectados

7. Organización del Equipo

Integrante	Modulo principal
García Aguilar José Iván	Lexer y tokens
Martínez Chavez Alexis	Parser y representación interna
Martínez Méndez Cedrik Alexis	Ensamblador de una y dos pasadas
Motta Reyes Emmanuel Alberto	Encoder IA-32, ModRM, SIB
Arcos Garduño Itzcoatl Ayax	Formato objeto y Linker

8. Dificultades Encontradas

Parsing de directivas con identificadores

La directiva SECTION .text fallaba porque .TEXT era tokenizado como identificador y el parser no tenia caso para ese tipo en parse_operand,

causando un loop infinito. Se resuelve agregando el caso faltante con avance forzado del token.

Direccionamiento simbólico en memoria

Instrucciones como `MOV [resultado_suma], EAX` fallaban porque `parse_memory` solo aceptaba registros o números como contenido de corchetes. Se resolvió agregando soporte para identificadores como direcciones de memoria.

Validación contra NASM

La comparacion directa contra NASM con archivos que usan `SECTION` mostro diferencias porque NASM genera encabezados de seccion adicionales en formato bin. La solución fue comparar instrucciones aisladas sin directivas de seccion, confirmando que los opcodes generados son correctos.

Conflictos de Git en trabajo en equipo

Al trabajar con múltiples Codespaces en paralelo se generaron conflictos al hacer push porque el repositorio remoto tenia commits mas recientes. Se resolvió con `git rebase --abort` seguido de `git push --force` para conservar los cambios locales correctos.

Separación de headers al integrar registros de 8 bits

Al agregar soporte para registros de 8 bits se colocaron accidentalmente las constantes `REG_AL` a `REG_BH` en `lexer.h` en lugar de `encoder.h`, rompiendo las definiciones de Lexer y Token. Se resuelve separando correctamente las responsabilidades de cada header.

9. Conclusiones

El proyecto logró implementar exitosamente un sistema completo de ensamblado y enlazado para un subconjunto de la arquitectura IA-32.

Los objetivos alcanzados fueron:

- Implementacion de un lexer y parser manuales sin herramientas automáticas
- Ensamblador funcional de una y dos pasadas con manejo de fixups

- Encoder IA-32 con soporte completo para ModRM, SIB, registros de 8 bits y MOVZX
- Formato objeto propio con secciones, símbolos y relaciones
- Mini linker capaz de enlazar múltiples módulos
- Salida hexadecimal legible con opción --hex
- Validación de correctitud byte a byte contra NASM

El desarrollo del proyecto permitió comprender en profundidad como funciona internamente la traducción de código ensamblador a código máquina, el rol de los archivos objeto como unidades de compilación independientes, y el proceso de enlazado que resuelve referencias entre módulos y genera el binario final.